

Xilinx FFT IP ve Zynq PS/PL Entegrasyonu ile FPGA Üzerinde FFT Gerçeklemesi

Baryonic Space FPGA/Digital Design Engineer Interview Project

7 Mayıs 2026

1 Giriş

Bu çalışma, Baryonic Space FPGA/Digital Design Engineer Interview Project kapsamında geliştirilen dört aşamalı FFT doğrulama akışını özetlemektedir. Amaç yalnızca Xilinx FFT IP'yi çalıştırmak değil; aynı zamanda leakage davranışını gözlemek, pencereleme etkisini hem zaman hem frekans domaininde doğrulamak ve son aşamada aynı veri yolunu Zynq PS/PL mimarisi içine yerleştirilerek yazılım kontrollü bir çevrebirim haline getirmektir.

Raporun akışı şu şekildedir:

- **Part 1:** Naive FFT veri yolu kurulmuş ve rectangular window kaynaklı spectral leakage davranışı gözlenmiştir.
- **Part 2:** Girişe time-domain Hamming window eklenmiş ve yan lob bastırma etkisi doğrulanmıştır.
- **Part 3:** Aynı pencereleme etkisi FFT çıkışında three-term frequency-domain işlem ile yeniden üretilmiştir.
- **Part 4:** DDS+FFT zinciri Zynq PS/PL sistemine bağlanmış, AXI-Lite register erişimleri ve bare-metal test yazılımı ile kart üzerinde doğrulanmıştır.

Bu kısaltılmış sürümde teori, modül açıklamaları ve log detayları sıkıştırılmış; buna karşılık görseller, waveform kanıtları, kritik karşılaştırma tabloları ve donanım çıktıları korunmuştur.

2 Teorik Arka Plan

Projede analiz edilen sinyal, sürekli zamandan ayırık zamana örneklenmiş kompleks üsteldir. FFT ise sonsuz uzunluklu bir dönüşüm değil, $N = 256$ örnekten oluşan sonlu bir frame üzerinde hesaplanan DFT'nin donanımda hızlı uygulanmış halidir. Bu nedenle giriş sinyalinin bin merkezine tam oturup oturmaması, spektrumun nasıl görüneceğini doğrudan belirlemektedir.

Bu çalışmada ortak parametreler

$$N = 256, \quad f_s = 200 \text{ MHz}$$

ve buna karşılık gelen FFT frekans çözünürlüğü

$$\Delta f = \frac{f_s}{N} = 781.25 \text{ kHz}$$

olarak kullanılmıştır. Bir giriş tonunun teorik bin konumu ise

$$k_0 = \frac{f_0 N}{f_s}$$

ile hesaplanır.

Bu hesap doğrudan Part 1–Part 4 sonuçlarını açıklar:

- 12.5 MHz için $k_0 = 16$ olduğundan sinyal FFT bin merkezine tam oturur.
- 10 MHz için $k_0 = 12.8$ olduğundan enerji tek bir bin yerine komşu binlere dağılır ve peak'ın 13 civarında görülmesi beklenir.

Rectangular window etkisi, sonlu frame kullanımı nedeniyle spektrumda leakage üretir. Bu durum özellikle off-bin tonlarda görünür hale gelir. Windowing işlemleri sinyal frekansını değiştirmez; değiştirdikleri şey spektral enerjinin dağılımıdır. Bu nedenle Part 2 ve Part 3'te peak bin aynı kalırken, yan lob seviyeleri ve ana lob şekli değişmektedir.

Tüm bölümlerde peak tespiti için FFT çıkışının büyüklük karesi kullanılmıştır:

$$|X[k]|^2 = \Re\{X[k]\}^2 + \Im\{X[k]\}^2$$

Bu tercih, karekök işlemine gerek bırakmadan maksimum binin güvenilir biçimde bulunmasını sağlar. Donanım açısından da daha ucuz ve doğrudandır.

Hamming window'un Part 2 ve Part 3 açısından önemli özelliği, zaman domenindeki etkisinin frekans domeninde üç terimli bir komşu-bin işlemi ile yeniden üretilebilmesidir. Bu çalışmada kullanılan eşdeğer ifade

$$Y[m] = 0.54X[m] - 0.23X[m-1] - 0.23X[m+1]$$

şeklinde. Böylece Part 2'de girişte yapılan pencereleme, Part 3'te FFT çıkışında uygulanmış ve iki yaklaşımın donanım üzerinde eşdeğer sonuç verdiği gösterilmiştir.

Waveform yorumlarında ortak kural şudur: anlık FFT bin bilgileri yalnızca ilgili `fft_out_valid` benzeri geçerli işaretler aktifken anlamlıdır; frame sonu özetleri ise yalnızca `peak_valid` ile birlikte yorumlanmalıdır. Bu ortak kural her waveform için tekrar edilmek yerine tüm raporda tek referans kabul edilmiştir.

3 Part 1: Naive FFT Implementation

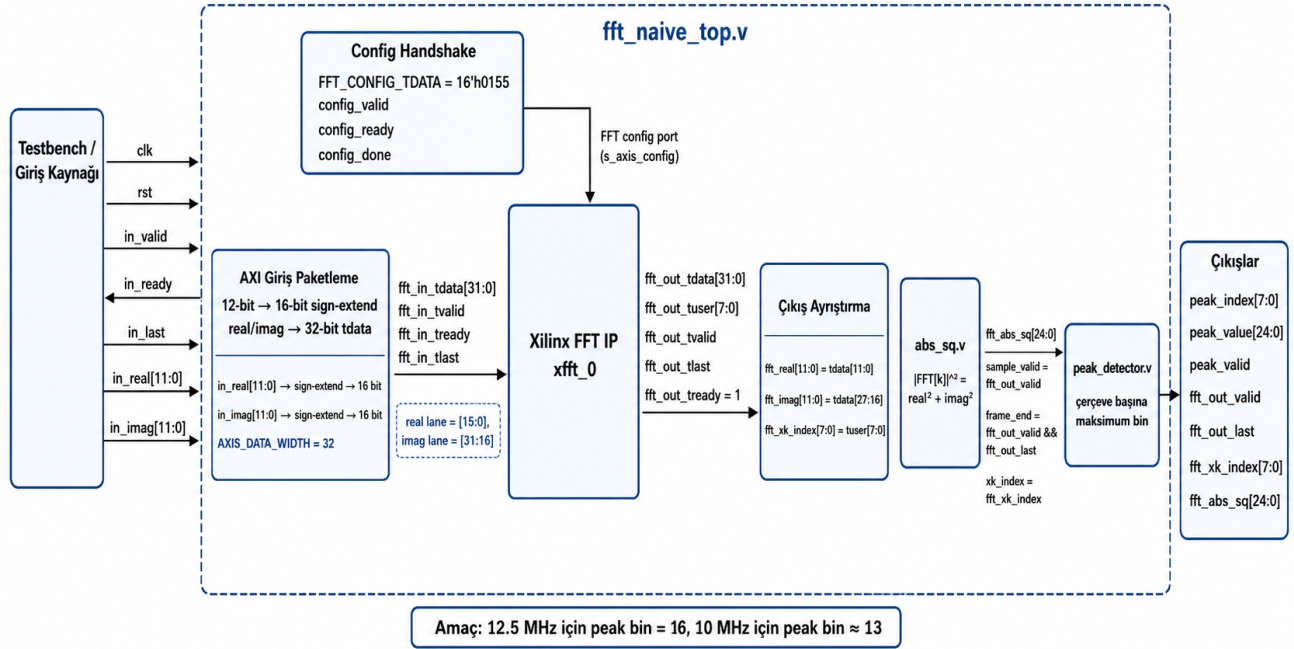
Part 1'de amaç, Xilinx FFT IP etrafında temel bir AXI-Stream veri yolu kurmak ve leakage davranışını gözlemektir. Testbench tarafından üretilen 12-bit real ve imag örnekleri, 16-bit lane hizalaması ile 32-bit `tdata` formatına paketlenmiş; FFT çıkışı `abs_sq` ve `peak_detector` zinciri ile işlenmiştir.

Parametre	Değer
Transform length	256
Input / output data width	12 bit
Data format	Fixed-point
Architecture	Pipelined Streaming I/O
Output ordering	Natural order
xk_index	Enabled
AXI input / output tdata width	32 bit
AXI tuser width	8 bit
FFT config word	16'h0155

Tablo 1: Part 1 için kullanılan temel Xilinx FFT IP yapılandırması

Modül yapısı üç ana blokta toplanmaktadır: `fft_naive_top.v` giriş örneklerini FFT IP'nin beklediği formata çevirir ve akışı yönetir; `abs_sq.v` her bin için büyüklük kare hesabını yapar; `peak_detector.v` ise frame boyunca gelen değerler içinden maksimumu seçer. Testbench tarafında iki ton kullanılmıştır: 12.5 MHz (bin merkezli) ve 10 MHz (off-bin).

Part 1 Veri Akışı Blok Diyagramı

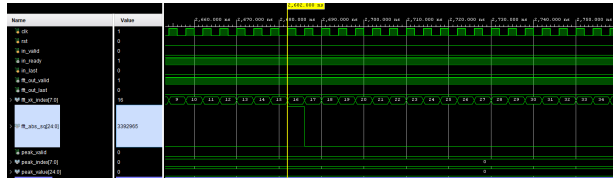


Şekil 1: Part 1 naive FFT veri yolu. Testbench kaynaklı kompleks örnekler FFT IP'ye gönderilir; çıkışta `abs_sq` ve `peak_detector` blokları ile tepe bin tespit edilir.

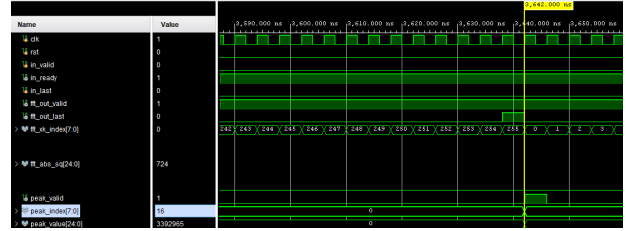
Frame	Frequency	Theoretical bin	Measured peak	Peak value	Comment
0	12.5 MHz	16.0	16	3392965	Bin-centered, minimum leakage
1	10.0 MHz	12.8	13	2968146	Off-bin, leakage observed

Tablo 2: Part 1 simülasyon sonuç özeti

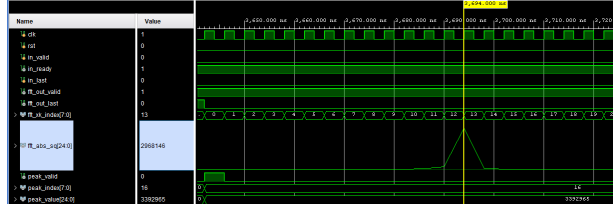
Part 1'in temel sonucu şudur: 12.5 MHz için `peak_index = 16` elde edilmiştir. 10 MHz için teorik bin 12.8 olduğundan `peak_index = 13` ölçülmüş ve komşu binlerdeki enerji dağılımı spectral leakage etkisini doğrulamıştır.



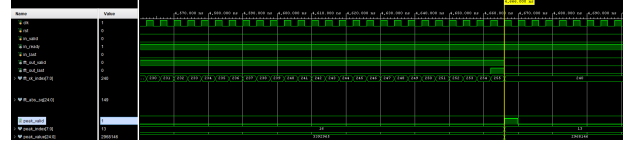
(a) 12.5 MHz anlık FFT çıkışı



(b) 12.5 MHz frame sonu peak üretimi



(c) 10 MHz leakage bölgesi



(d) 10 MHz frame sonu peak üretimi

Şekil 2: Part 1 waveform özeti. 12.5 MHz ton tek bin etrafında yoğunlaşırken 10 MHz girişte enerji komşu binlere dağılmıştır. Peak detector her iki durumda da beklenen baskın bin konumunu üretmiştir.

Bu bölümde elde edilen sonuç, FFT veri yolunun temel olarak doğru kurulduğunu ve teorik leakage davranışının donanım simülasyonunda açıkça gözlemlendiğini göstermektedir.

4 Part 2: Time-Domain Hamming Window

Part 2'de aynı FFT veri yolu korunmuş, yalnızca girişe Hamming window eklenmiştir. Amaç peak bin konumunu değiştirmeden leakage yan loblarını bastırmaktır. Kullanılan temel ifade

$$w_H[n] = 0.54 - 0.46 \cos\left(\frac{2\pi n}{N}\right), \quad x_w[n] = x[n]w_H[n]$$

şeklinde. Hamming katsayıları Python tarafında üretilmiş, Q1.15 formata çevrilmiş ve Verilog içinde bellek dosyasından okunmuştur.

`hamming_window.v` bloğu her input örneğini frame indeksine karşılık gelen pencere katsayısı ile çarpar ve sonucu FFT IP'ye uygun genişlikte yeniden ölçekler. Testbench akışı Part 1 ile aynıdır; bu sayede fark yalnızca pencereleme etkisinden kaynaklanmaktadır.

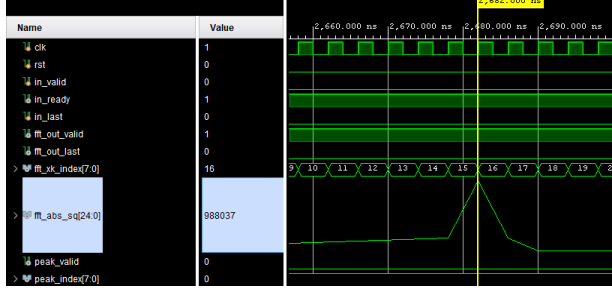
Signal	Method	Theoretical bin	Peak bin	Peak value	Observation
12.5 MHz	Part 1 Naive FFT	16.0	16	3392965	Energy concentrated in one bin
12.5 MHz	Part 2 Hamming FFT	16.0	16	988037	Peak preserved, main lobe spread to 15–17
10 MHz	Part 1 Naive FFT	12.8	13	2968146	Visible leakage
10 MHz	Part 2 Hamming FFT	12.8	13	926773	Far side lobes suppressed

Tablo 3: Part 1 ve Part 2 sonuçlarının özeti

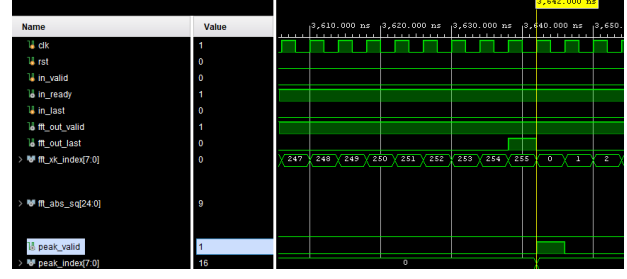
Bin	Part 1 abs_sq	Part 2 abs_sq	Observation
10	15426	4	Strong suppression
11	36941	549	Suppressed
12	185434	342325	Main lobe region
13	2968146	926773	Peak preserved
14	82645	76869	Main lobe region
15	24484	45	Suppressed
16	11642	8	Suppressed
17	6740	18	Suppressed
18	4394	18	Suppressed

Tablo 4: 10 MHz için leakage karşılaştırması: Part 1 vs Part 2

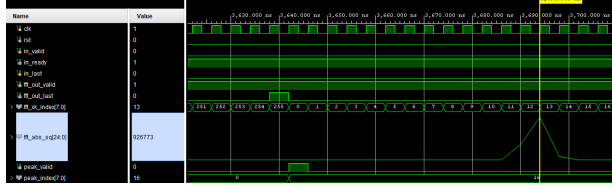
Hamming window peak frekans konumunu değiştirmemiştir; 12.5 MHz için peak yine 16, 10 MHz için peak yine 13 olmuştur. Ancak uzak yan loblar Part 1'e göre belirgin biçimde bastırılmış, enerji ana lob çevresinde daha kontrollü toplanmıştır.



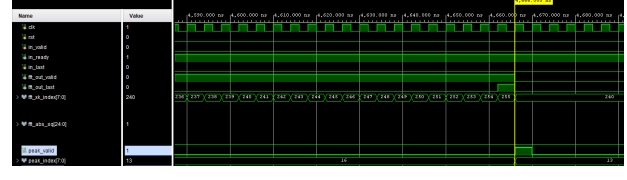
(a) 12.5 MHz anlık windowed FFT çıkışı



(b) 12.5 MHz frame sonu peak üretimi



(c) 10 MHz anlık windowed leakage davranışı



(d) 10 MHz frame sonu peak üretimi

Şekil 3: Part 2 waveform özeti. Peak bin değişmezken enerji daha geniş ama kontrollü bir ana lobe dağılmış, uzak leakage bileşenleri bastırılmıştır.

Bu bölümde elde edilen sonuç, time-domain windowing'in leakage kontrolü için etkili olduğunu ve özellikle off-bin tonlarda spektrumun daha düzenli hale geldiğini göstermektedir.

5 Part 3: Frequency-Domain Three-Term Windowing

Part 3'te Hamming window etkisi FFT girişinde değil, FFT çıkışında üç terimli bir komşu-bin işlemi ile üretilmiştir:

$$Y[m] = 0.54X[m] - 0.23X[m - 1] - 0.23X[m + 1]$$

Bu yaklaşım, time-domain Hamming window'un frekans domenindeki eşdeğeridir. **three_term.v** modülü önce tüm frame'i yakalayıp, ardından capture/sweep yapısı ile her bin için yeni değeri hesaplar. Circular boundary koşulları nedeniyle $m - 1$ ve $m + 1$ indeksleri modüler olarak ele alınır.

FSM mantığı kısa olarak üç aşamada özetlenebilir: **S_CAPTURE** aşamasında FFT binleri belleğe alınır; **S_SWEEP** aşamasında üç terimli yeniden ağırlıklandırma uygulanır; **S_DONE** aşamasında yeni frame beklenir. Bu yapı, Part 2 ile doğrudan sayısal karşılaştırma yapmayı mümkün kılmıştır.

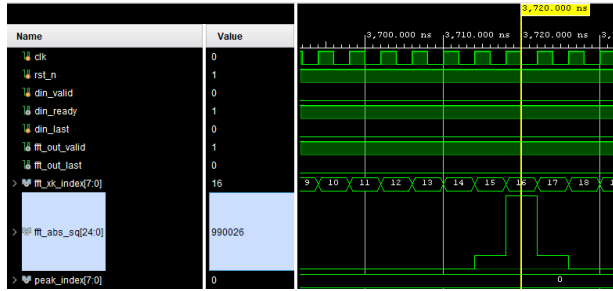
Frame	Input	Theoretical bin	Peak bin	Peak value	Observation
0	12.5 MHz freqwin	16.0	16	990026	Hamming main lobe around 15–17
1	10.0 MHz freqwin	12.8	13	927197	Far side lobes suppressed

Tablo 5: Part 3 frequency-domain windowing sonuç özeti

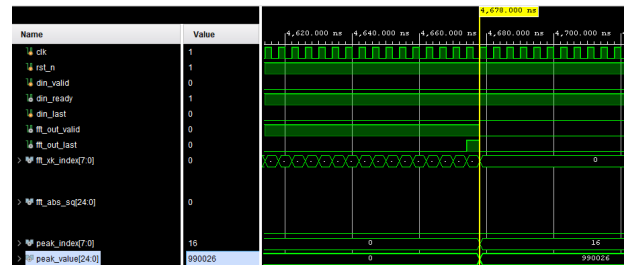
Input	Part 2 peak	Part 3 peak	Part 2 peak value	Part 3 peak value	Observation
12.5 MHz	16	16	988037	990026	Very close, same dominant bin
10.0 MHz	13	13	926773	927197	Frequency-domain equivalence confirmed

Tablo 6: Part 2 ve Part 3 karşılaştırması

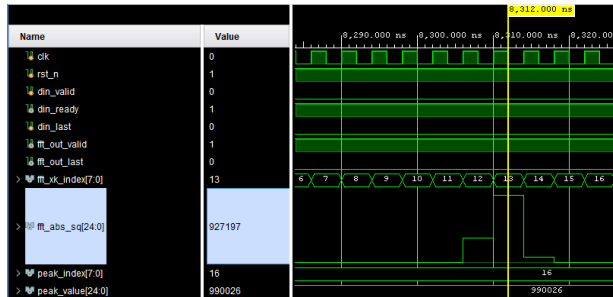
Part 3'ün ana çıktısı, Part 2'de zaman domeninde uygulanan pencereleme etkisinin FFT çıkışında yeniden üretilebildiğini göstermesidir. Peak bin değerleri Part 2 ile aynıdır; peak value ve total energy değerleri ise sabit nokta yuvarlama farkları dışında oldukça yakındır.



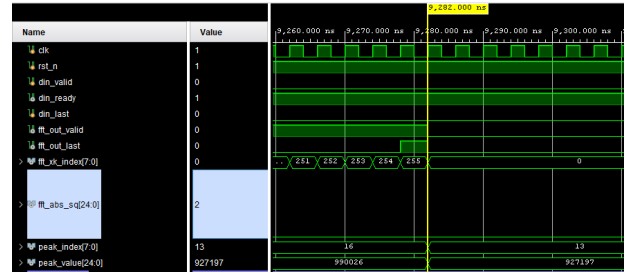
(a) 12.5 MHz anlık three-term çıkışı



(b) 12.5 MHz frame sonu peak üretimi



(c) 10 MHz anlık three-term leakage davranışı



(d) 10 MHz frame sonu peak üretimi

Şekil 4: Part 3 waveform özeti. 12.5 MHz için peak 16'da, 10 MHz için peak 13'te kalmıştır; bu da Part 2 ile tutarlı frekans domeni pencereleme davranışını göstermektedir.

Bu bölümde elde edilen sonuç, time-domain Hamming window ile frequency-domain three-term windowing'in donanım üzerinde tutarlı sonuç verdiğini açıkça göstermektedir.

6 Part 4: Zynq PS/PL Integration

Part 4'te önceki bölümlerde doğrulanan FFT zinciri Zynq PS/PL mimarisi içine taşınmıştır. Amaç, PS tarafındaki yazılımın PL içinde çalışan DDS+FFT veri yolunu AXI-Lite register erişimleri ile kontrol edebildiğini ve sonuçları kart üzerinde geri okuyabildiğini göstermektir.

Block	Type	Role
<code>processing_system7_0</code>	ZYNQ7 PS	ARM çekirdekleri, DDR/MIO ve FCLK_CLK0 üretimi
<code>rst_ps7_0_200M</code>	Reset IP	Senkron <code>peripheral_aresetn</code> üretimi
<code>axi_interconnect_0</code>	AXI Interconnect	<code>M_AXI_GP0</code> ile <code>fft_pl_ps_0/S_AXI</code> arasında yönlendirme
<code>fft_pl_ps_0</code>	Module Reference	AXI-Lite register bankası, DDS, FFT ve peak tespit zinciri
<code>ila_0 / Constant / Concat</code>	Debug utilities	İç sinyalleri izleme ve <code>peak_index</code> zero-padding

PS tarafındaki M_AXI_GP0 master portu, axi_interconnect_0 üzerinden fft_pl_ps_0/S_AXI slave portuna bağlanmıştır. Ortak saat olarak FCLK_CLK0 = 200 MHz kullanılmış, reset hattı ise FCLK_RESET0_N çıkışından Processor System Reset bloğuna verilerek senkron peripheral_aresetn hattına dönüştürülmüştür.

Şekil 5: Part 4 block design. PS tarafı AXI master, PL tarafındaki `fft_pl_ps_0` ise AXI-Lite slave çevrebirimi olarak çalışmaktadır.

Offset	Register	Access	Description
0x00	DDS_PINC	RW	PS tarafından yazılan DDS phase increment
0x04	PEAK_STATUS	RO / W1C	[7:0] peak index, [8] sticky valid
0x08	PEAK_VALUE	RO	Peak abs_sq değeri
0x0C	BUILD_INFO	RO	Versiyon/imza bilgisi

DDS kontrolü için kullanılan temel denklem

$$\text{PINC} = \text{round} \left(\frac{f_{\text{desired}}}{f_s} 2^{32} \right)$$

şeklindedir. Bu nedenle 10 MHz için

$$\text{PINC} = \text{round} \left(\frac{10 \times 10^6}{200 \times 10^6} 2^{32} \right) = 0x0CCCCCD$$

elde edilir. Yazılım tarafı bu değeri DDS_PINC register'ına yazar, sticky valid bitini temizler, yeni peak sonucunu bekler ve ardından PEAK_STATUS ile PEAK_VALUE register'larını okur.

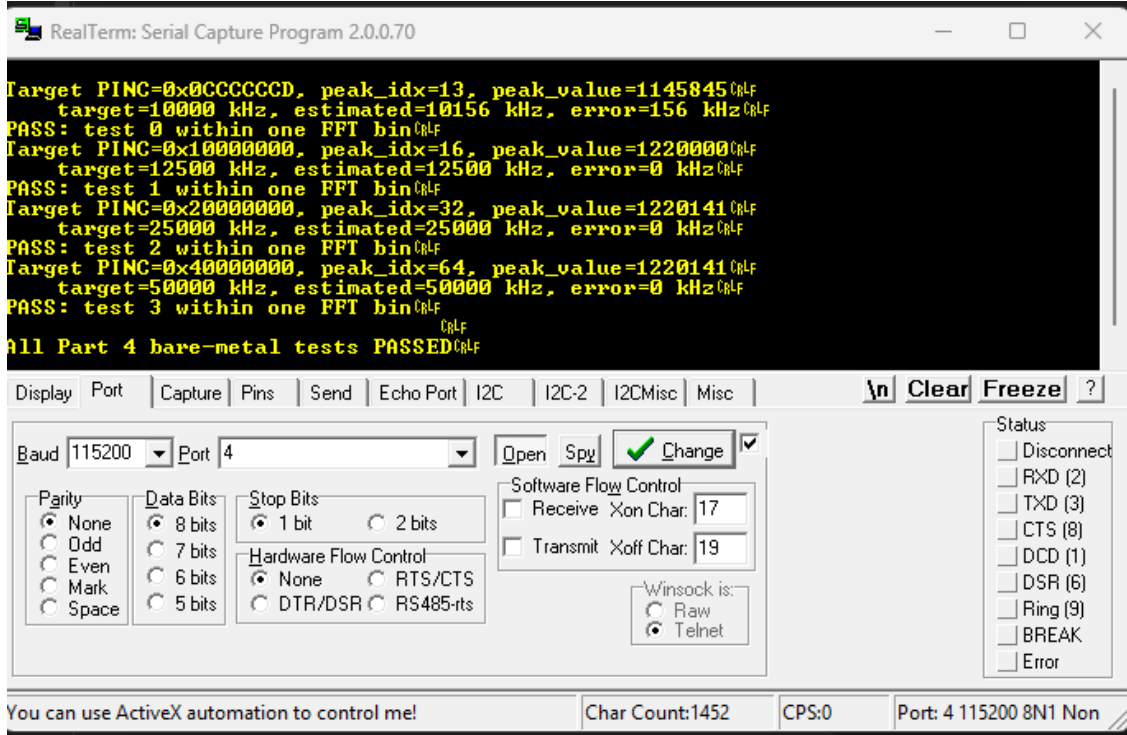
```
write_reg(DDS_PINC_OFFSET, pinc);
write_reg(PEAK_STATUS_OFFSET, 0x100);
do {
    status = read_reg(PEAK_STATUS_OFFSET);
} while ((status & 0x100) == 0);
peak_idx = status & 0xFF;
peak_value = read_reg(PEAK_VALUE_OFFSET);
```

ILA debug ağı fonksiyonel veri yolunun parçası değildir; yalnızca gözlem amaçlıdır. `peak_index_dbg[7:0]`, `peak_valid_dbg` ve `dds_pinc_dbg[31:0]` ILA'ya bağlanmış, `peak_index_dbg` ise Constant ve Concat blokları ile 32 bit'e sıfır genişletilmiştir.

Target	PINC (hex)	Peak Index	Estimate	Error	Peak Value	Result
10 MHz	0x0CCCCCD	13	10.156 MHz	156 kHz	1 144 900	PASS
12.5 MHz	0x10000000	16	12.5 MHz	0 kHz	1 219 301	PASS
25 MHz	0x20000000	32	25 MHz	0 kHz	1 222 065	PASS
50 MHz	0x40000000	64	50 MHz	0 kHz	1 220 785	PASS

Tablo 9: Part 4 donanım üzerinde ölçülen sonuçlar

PS tarafı DDS_PINC register'ına yazarak hedef frekansı belirlemiş, PL tarafı DDS+FFT zinciri ile peak index üretmiş ve sonuçlar AXI-Lite register'ları üzerinden geri okunmuştur. 10 MHz, 12.5 MHz, 25 MHz ve 50 MHz testlerinin tamamı bir FFT bininden küçük hata ile PASS olmuştur.



Şekil 6: Part 4 UART/RealTerm çıktısı. Yazılımın PL register'larına erişebildiği ve peak sonuçlarını kart üzerinden okuduğu doğrudan görülmektedir.

UART/RealTerm çıktısı, yazılımın PL register alanına erişebildiğini, PINC yazma işleminin doğru çalıştığını ve peak index bilgisinin beklenen binlere karşılık geldiğini göstermektedir. Özellikle 10 MHz için $peak_index = 13$ sonucu teorik $k = 12.8$ hesabı ile uyumludur; 12.5 MHz, 25 MHz ve 50 MHz ise bin merkezlerine tam oturduğu için hata 0 kHz çıkmıştır.

Bu bölümde elde edilen sonuç, tasarımın yalnızca simülasyonda değil gerçek Zybo Z7-20 kartı üzerinde de kapalı çevrim biçimde doğrulandığını göstermektedir.

7 Genel Sonuçlar ve Tartışma

Part 1, Part 2 ve Part 3 birlikte değerlendirildiğinde, peak bin değerlerinin pencereleme yönteminden bağımsız olarak korunduğu görülmektedir. Beklenen de budur; çünkü windowing işlemleri sinyal frekansını değiştirmez, yalnızca spektral enerji dağılımını yeniden şekillendirir. Fark yaratan nokta, leakage seviyeleri ve ana lob davranışdır.

Method	Applied processing	12.5 MHz peak	10 MHz peak	Main observation
Part 1 Naive FFT	Rectangular window effect only	16	13	Leakage observed for 10 MHz
Part 2 Time-domain Hamming	Hamming window at FFT input	16	13	Far side lobes suppressed, main lobe widened
Part 3 Frequency-domain windowing	Three-term processing at FFT output	16	13	Very close to Part 2, theoretical equivalence verified

Tablo 10: Part 1–Part 3 genel karşılaştırması

Bu karşılaştırma birkaç temel sonucu öne çıkarmaktadır:

- Part 1, temel FFT davranışını ve rectangular window kaynaklı leakage etkisini açık biçimde göstermiştir.
- Part 2, time-domain Hamming window ile özellikle off-bin tonlarda yan lobları bastırmıştır.
- Part 3, aynı pencereleme etkisini FFT çıkışında frequency-domain three-term işlem ile üretmiştir.
- Part 2 ve Part 3 sonuçlarının çok yakın olması, teorik eşdeğerliğin donanım üzerinde doğru kurulduğunu göstermektedir.
- Peak bin değerlerinin sabit kalması, pencerelemenin frekansı değil enerji dağılımını değiştirdiğini doğrulamaktadır.

Part 4 sonuçları bu tabloyu sistem düzeyinde tamamlamaktadır. UART/RealTerm çıktısı, yazılımın PL register alanına erişebildiğini göstermiştir. PINC yazıldıktan sonra `peak_index` değerlerinin beklenen binlere karşılık geldiği doğrulanmıştır. 10 MHz için `peak_index = 13` sonucu teorik $k = 12.8$ ile uyumludur. 12.5 MHz, 25 MHz ve 50 MHz testlerinde hata 0 kHz çıkması ise bin merkezli tonların beklenen davranışını yansıtmaktadır. Tüm testlerin PASS olması, tasarımın yalnızca simülasyonda değil gerçek kart üzerinde de doğrulandığını göstermektedir.

8 Sonuç

Bu çalışma boyunca FFT IP tabanlı bir DSP veri yolu önce simülasyon ortamında kurulmuş, ardından windowing yaklaşımlarıyla genişletilmiş ve son aşamada Zynq PS/PL mimarisi içine entegre edilmiştir. Part 1 temel FFT ve leakage davranışını, Part 2 zaman domeninde Hamming window etkisini, Part 3 aynı etkinin frekans domenindeki three-term eşdeğerini, Part 4 ise PS kontrollü gerçek donanım doğrulamasını göstermiştir. Böylece proje; DSP teorisi, fixed-point FPGA implementasyonu, AXI-Stream ve AXI-Lite veri yolu kullanımı ile gerçek kart doğrulamasını bir araya getiren bütünlüklü bir FPGA DSP tasarımı ortaya koymuştur.

Kısaltılmış bu sürümde teori ve modül detayları sıkıştırılmış olsa da, görseller, waveform’lar, karşılaştırma tabloları ve donanım çıktıları korunmuştur. Bu nedenle rapor, hem tasarım kararlarını hem de bu kararların sayısal ve deneysel doğrulamasını kompakt ama yeterli düzeyde sunmaktadır.

9 Kaynakça ve Ekler

9.1 Kaynakça

Aşağıdaki tablo, proje boyunca yararlanılan temel teknik kaynakları ve bu kaynakların kullanım amaçlarını özetlemektedir.

No	Kaynak	Kullanım Amacı	Bağlantı / Not
1	Baryonic Space FPGA/Digital Design Engineer Interview Project	Projenin ana görev tanımı, Part 1–Part 4 kapsamı, FFT/windowing beklentileri, Zynq PS/PL bonus isterleri ve doğrulama hedefleri için temel referans olarak kullanılmıştır.	Kullanıcıya sağlanan proje PDF dokümanı.
2	Xilinx FFT IP / Vivado Generated Instantiation	Xilinx FFT IP’nin 256-point, fixed-point, AXI-Stream tabanlı kullanımı; <code>tdata/tuser</code> portları, <code>xk_index</code> çıkışı, config kanalı ve natural-order FFT çıkışı için referans alınmıştır.	Vivado tarafından üretilen <code>xfft_0</code> IP wrapper/instantiation dosyaları ve IP configuration ekranları.
3	AMBA AXI4-Stream Protocol Specification	AXI-Stream <code>valid/ready</code> handshake, <code>tdata</code> , <code>tlast</code> ve <code>tuser</code> sinyallerinin yorumlanması için temel protokol referansı olarak kullanılmıştır.	Scribd üzerinde paylaşılan AMBA AXI4-Stream v1.0 protokol dokümanı.
4	Mehmet Burak Aykenar YouTube Channel	Vivado, FPGA tasarım akışı, Zynq SoC, AXI tabanlı özel IP oluşturma ve donanım/yazılım entegrasyonu konularında genel öğrenme ve uygulama desteği için kullanılmıştır.	YouTube kanalındaki Vivado ve Zynq odaklı eğitim içerikleri.
5	Mehmet Burak Aykenar GitHub Repository — <code>apis_anatolia</code>	AXI-Lite/custom IP örnekleri, Vivado IP paketleme yaklaşımı, register tabanlı çevrebirim mantığı ve test/verification örnekleri için referans alınmıştır. Özellikle AXI4-Lite slave register mantığı ve custom IP yapısının anlaşılmasında faydalanılmıştır.	GitHub deposu: <code>mbaykenar/apis_anatolia</code> .
6	Mehmet Burak Aykenar — Custom AXI4-Lite / AXI4 IP Examples	Vivado’da AXI4-Lite arayüzlü custom IP oluşturma, register bankası mantığı ve AXI-Lite tabanlı PS/PL haberleşme yapısını anlamak için kullanılmıştır.	İlgili blog yazısı ile <code>axifull_ip</code> ve <code>axilite_uvwm</code> örnek klasörleri.

No	Kaynak	Kullanım Amacı	Bağlantı / Not
7	Farbius DSP Xilinx IP GitHub Repository	Xilinx IP çekirdekleriyle DSP uygulamaları, FFT IP kullanımı ve proje kapsamındaki genel DSP-on-FPGA yaklaşımı için referans alınmıştır.	Farbius tarafından paylaşılan DSP-on-FPGA örnek deposu.
8	Embedded.com — DSP Tricks: Frequency-Domain Windowing	Part 3'te kullanılan Hamming window'un frekans domenindeki üç terimli eşdeğerinin teorik temeli için kullanılmıştır.	https://www.embedded.com/dsp-tricks-frequency-domain-windowing/
9	Richard G. Lyons — <i>Understanding Digital Signal Processing</i>	Frequency-domain windowing kavramı ve three-term windowing eşdeğerliği için teorik arka plan olarak kullanılmıştır.	Embedded.com'daki frequency-domain windowing yazısı, <i>Understanding Digital Signal Processing</i> kitabındaki ilgili DSP tricks içeriğine dayanmaktadır.
10	Python / NumPy Reference Model	Kompleks ton üretimi, FFT referans hesapları, Hamming window katsayı üretimi ve fixed-point .mem dosyalarının oluşturulması için kullanılmıştır.	Proje kapsamında yazılan reference_fft.py ve generate_hamming_mem.py dosyaları.
11	Vivado, Vitis ve RealTerm Araçları	Vivado sentez/simülasyon/IP entegrasyonu, Vitis bare-metal PS yazılımı ve RealTerm üzerinden UART çıktısının doğrulanması için kullanılmıştır.	Kullanılan geliştirme ve doğrulama araçları.

9.2 Ekler

Aşağıdaki tablo, rapor kapsamında kullanılan temel dosya, çıktı ve yardımcı içerikleri özetlemektedir.

Ek No	İçerik	Açıklama
Ek A	Python Referans Dosyaları	reference_fft.py ile 12.5 MHz ve 10 MHz kompleks tonlar üretilmiş, Python FFT ile beklenen peak bin değerleri doğrulanmıştır. generate_hamming_mem.py ile Hamming katsayıları Q1.15 formatında hamming_q1_15.mem dosyasına yazılmıştır.
Ek B	Verilog RTL Dosyaları	fft_naive_top.v , fft_windowed_top.v , three_term.v , hamming_window.v , abs_sq.v , peak_detector.v ve fft_pl_ps_top.v dosyaları tasarımın temel RTL modüllerini oluşturmaktadır.
Ek C	SystemVerilog Testbench Dosyaları	tb_fft_naive.sv , tb_fft_windowed.sv ve tb_fft_freqwin.sv dosyaları Part 1, Part 2 ve Part 3 simülasyon doğrulamalarında kullanılmıştır.
Ek D	Bellek Dosyaları	tone_12p5MHz_re.mem , tone_12p5MHz_im.mem , tone_10MHz_re.mem , tone_10MHz_im.mem ve hamming_q1_15.mem dosyaları testbench ve RTL simülasyonlarında kullanılmıştır.
Ek E	Vivado Block Design	Part 4 kapsamında Zynq PS, AXI Interconnect, fft_pl_ps_0 module reference, ILA, Constant ve Concat bloklarını içeren system.bd tasarımı kullanılmıştır.
Ek F	Vitis Bare-Metal Yazılımı	PS tarafından DDS_PINC register'ına frekans ayarı yazan, PEAK_STATUS ve PEAK_VALUE register'larını okuyarak FFT peak sonucunu doğrulayan C kodu kullanılmıştır.

Ek No	İçerik	Açıklama
Ek G	UART / RealTerm Donanım Çıktısı	Zybo Z7-20 üzerinde çalıştırılan Vitis uygulamasının 10 MHz, 12.5 MHz, 25 MHz ve 50 MHz testlerini PASS verdiğini gösteren RealTerm çıktısı rapora eklenmiştir.
Ek H	Waveform Görselleri	Part 1, Part 2 ve Part 3 için <code>fft_out_valid</code> , <code>fft_xk_index</code> , <code>fft_abs_sq</code> , <code>peak_valid</code> ve <code>peak_index</code> sinyallerini gösteren Vivado waveform ekran görüntüleri kullanılmıştır.

9.3 AI Kullanım Beyanı

Bu raporun hazırlanması sürecinde yapay zekâ destekli araçlardan yardımcı kaynak olarak faydalanılmıştır. AI desteği; Verilog/SystemVerilog kodlarının okunabilirliğinin artırılması, hata ayıklama sırasında olası nedenlerin tartışılması, Vitis/Vivado kullanım adımlarının düzenlenmesi, rapor dilinin sadeleştirilmesi, teknik açıklamaların rapor formatına dönüştürülmesi ve tablo/başlık yapısının iyileştirilmesi amacıyla kullanılmıştır.

AI araçları, tasarım kararlarının tek kaynağı olarak kullanılmamıştır. FFT IP yapılandırması, test-bench senaryoları, simülasyon sonuçları, waveform yorumları, AXI-Lite register map, Vitis bare-metal testleri ve donanım çıktıları kullanıcı tarafından Vivado, Vitis ve kart üzerinde doğrulanmıştır. Rapor içindeki sayısal sonuçlar, simülasyon logları, waveform ekran görüntüleri ve UART çıktıları proje kapsamında elde edilen gerçek doğrulama verilerine dayanmaktadır.

AI kullanımı özellikle şu alanlarda sınırlı kalmıştır:

- Kod bloklarının daha okunabilir hale getirilmesi
- Hata nedenlerinin yorumlanması
- AXI-Stream ve AXI-Lite sinyal açıklamalarının sadeleştirilmesi
- Matematiksel ifadelerin rapor formatına dönüştürülmesi
- Raporun bölüm yapısı, tablo düzeni ve sonuç yorumlarının iyileştirilmesi
- Türkçe teknik metnin daha anlaşılır ve tutarlı hale getirilmesi

Sonuç olarak AI, projede yardımcı bir düzenleme ve açıklama aracı olarak kullanılmış; tasarımın fonksiyonel doğrulaması ise simülasyon, waveform analizi ve Zybo Z7-20 üzerinde alınan gerçek UART çıktıları ile yapılmıştır. Bu nedenle rapordaki teknik sonuçların sorumluluğu kullanıcıya ait olup, AI yalnızca destekleyici düzenleme ve açıklama aracı olarak kullanılmıştır.